

Practice: 1

Task 1.1

a) 1) ~~Because~~ EmpID, SSN, Email ~~are~~ are unique identifiers, so our superkeys can be: (EmpID), (SSN), (Email), (EmpID, Name), (SSN, Department), (EmpID, SSN)

2) candidate keys: EmpID, SSN, Email

3) I'd choose EmpID, because it's immutable, short and strictly internal to the company

4) Yes it can be, ~~sometime~~ sometimes in company every department has ~~their~~ their own number, so employers from departments may share the same phone.

B) 1) Student Id, CourseCode, Section, Semester, Year.

2) Student Id identifies who is taking the course. CourseCode / Section identifies

which class section is being taken.
Semester/Year: distinguishes when the course was taken, since a student can repeat a course across different terms.

3) None, There are no other alternative candidate key.

Task 1.2.

1) Student. Advisor ID references Professor. Prof ID.

2) Course. Department code references Department. Dept code

3) Department. Chair ID references Professor. Prof ID

4) Enrollment. Student ID references Student. Student ID

5) Enrollment. Course ID references Course. Course ID

Task 2.1

1) Strong Entities: Patient, Doctor, Department, Hospital room.

Weak entities: Appointment, Prescription.

2) Patient
First Name
Last Name
Address
City
State
Zip
Phone
Email
Date of Birth
Gender
Race
Ethnicity
Religion
Marital Status
Occupation
Education
Income
Health Insurance
Medical History
Allergies
Current Medications
Vital Signs
Lab Results
Immunization
Referral
Referral Date
Referral From
Referral To
Referral Reason
Referral Status
Referral Date
Referral From
Referral To
Referral Reason
Referral Status

Doctor
First Name
Last Name
Address
City
State
Zip
Phone
Email
Date of Birth
Gender
Race
Ethnicity
Religion
Marital Status
Occupation
Education
Income
Health Insurance
Medical History
Allergies
Current Medications
Vital Signs
Lab Results
Immunization
Referral
Referral Date
Referral From
Referral To
Referral Reason
Referral Status

Department
Name
Address
City
State
Zip
Phone
Email
Date of Birth
Gender
Race
Ethnicity
Religion
Marital Status
Occupation
Education
Income
Health Insurance
Medical History
Allergies
Current Medications
Vital Signs
Lab Results
Immunization
Referral
Referral Date
Referral From
Referral To
Referral Reason
Referral Status

Appointment
Patient ID
Doctor ID
Department ID
Date
Time
Room
Status
Notes

Prescription
Patient ID
Doctor ID
Department ID
Date
Time
Room
Status
Notes

3) "Depart
"Docto
"Patie
"Patie

2) Patient: PatientID (Simple, PK), Name (Composite: First, last), birth dates (Simple), Address (Composite: street, city, state, zip), Phone (Multi-valued), Insurance Info (Composite/simple).

Doctor: DoctorID (Simple, PK), Name (Composite), Specialization (Multi-valued), Phone (Multi-valued), office location (simple).

Department: Dept code (Simple, PK), Name (Simple), location (simple).

Appointment: Appointment datetime (Part of PK), Purpose (simple), Notes (Derived/simple).

Prescription: prescription ID (PK), Dosage (simple), ~~Notes (Derived/simple)~~, Instructions (simple).

Hospital room: Room number, Dept code (Composite PK, weak entity concept), Room type (simple).

3) "Department" to "Hospital Room": 1:N

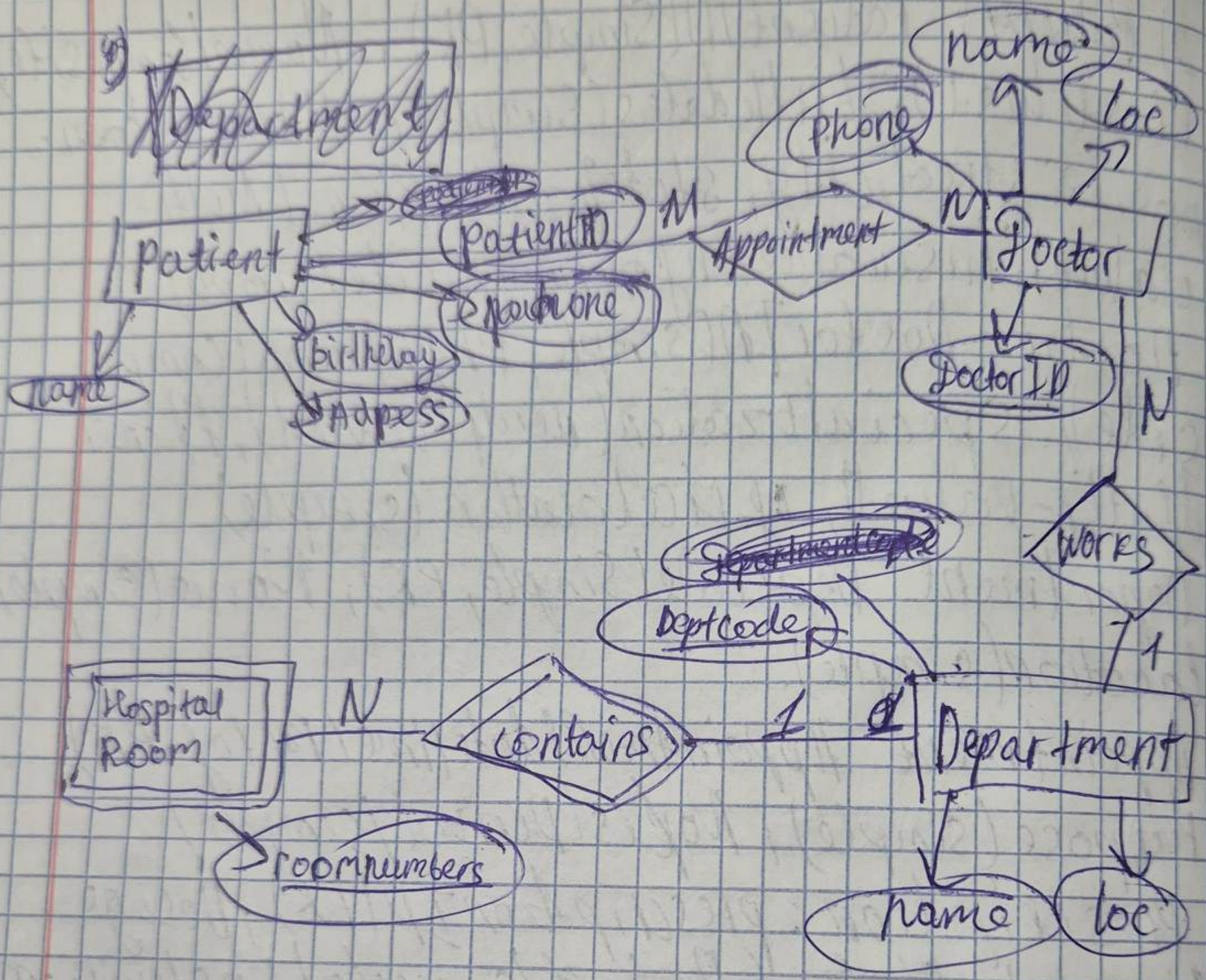
"Doctor" to "Department": N:1

"Patient" to "Doctor": M:N (via appointment)

"Patient" to "Doctor": M:N (via Prescription)

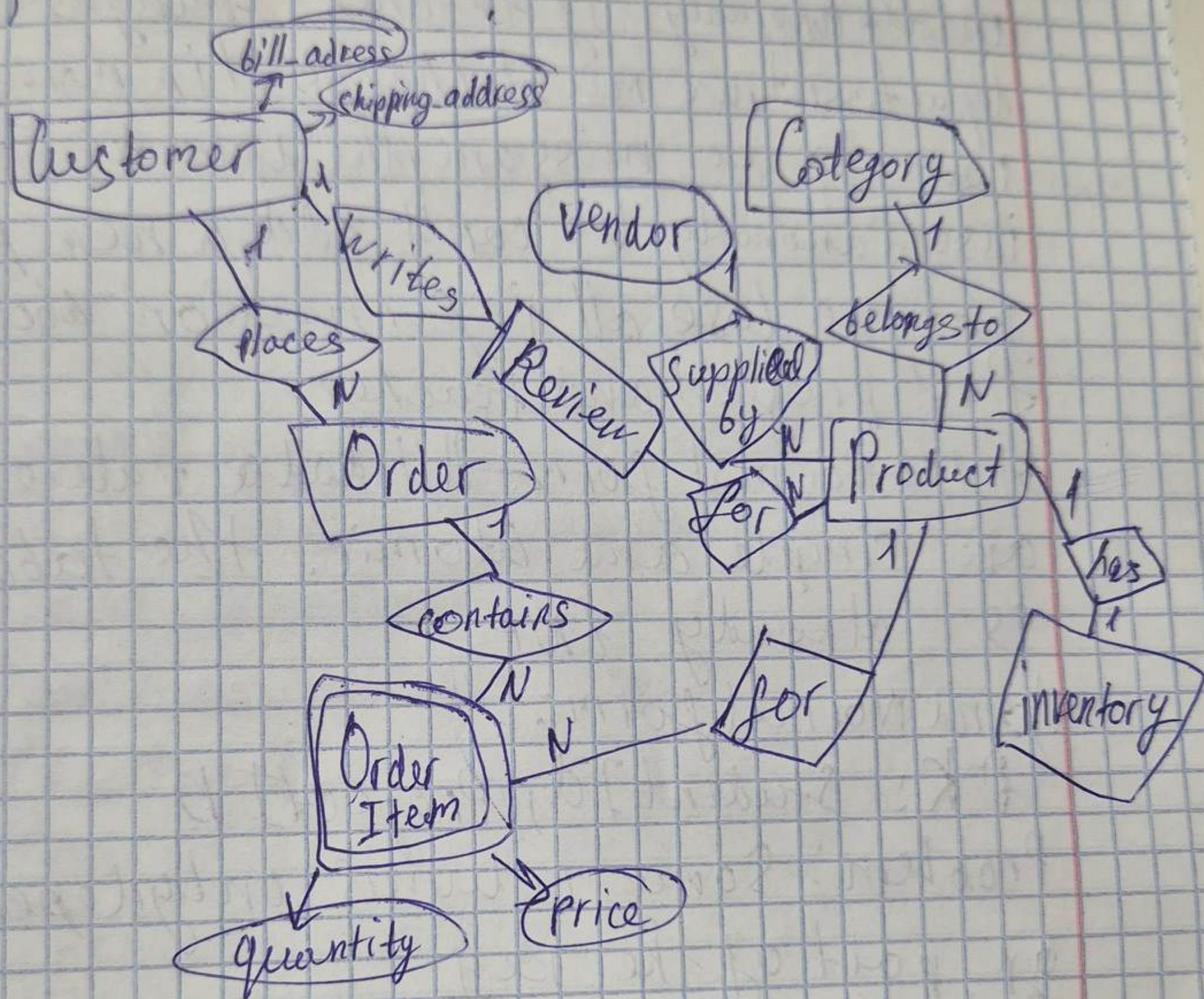
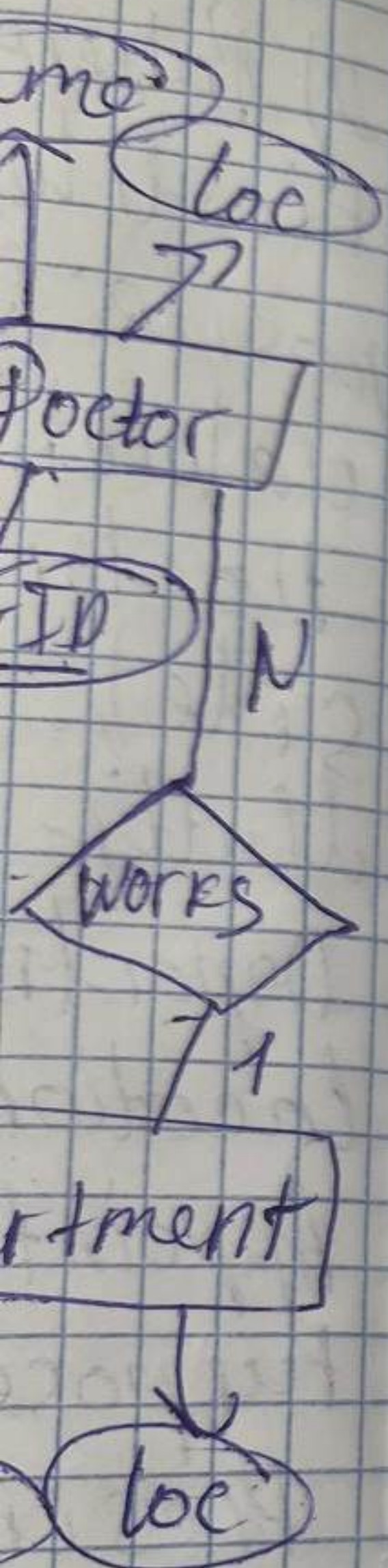
2.2)

cust



4.1 1)

2) Redu
and pro



- 4.1
- 1) StudentID $\Rightarrow$ (Studentname, studentmajor)
 - ProjectID $\Rightarrow$ (Projtitle, Projtype, Suis ID, Startdate, Enddate)
 - SupervisorID $\Rightarrow$ (Supervisorname, Supervisor Dept)
 - (StudID, projectID) $\Rightarrow$ (Role, Hours worked)
 - 2) Redundancy: We write the same student and project names many times.

update anomaly: If teacher changes their department, we must change it in many rows. If we miss one, the data is wrong.
insert anomaly: We can't add a new project, we lose all information about that project and teacher

3) 1st Normal form: All data values are simple and atomic. The table is already 1NF.

4) 2nd Normal form:

PK: StudentId, ProjectId

Problem: Some columns only depend on part of the key

Solution: Split into 3 tables:

Student, project, studentproject Assignment

5) 3rd Normal form:

Problem: Teacher details

SupervisorId, not ~~to~~ directly on the project.

Solution
a ne
stud

4.2

1) Pri

can fo

ent ti

Course

2) Fun

Sto

Co

In

IT

[Co

3) The

some

of the
key.

Solution: Move teachers details to a new table.

Student, Supervisor, project, Studprog Assignment.

4.2

1) Primary key: The trick is that a student can take different courses at different times. The primary key is (StudentID, CourseID, Time slot)

2) Functional Dependencies.

StudentID $\rightarrow$ Student Major

CourseID $\rightarrow$ Course Name

InstructorID $\rightarrow$ Instructor name

(Time slot, Room) $\rightarrow$ Building

(CourseID, Time slot) $\rightarrow$ (InstructorID, Room)

3) The table is not in BCNF because some columns depend on small parts of the data, not on the full superkey.

4) decomposing to BCNF by splitting into small tables:

Student (Student ID, Student Name)

Course (Course ID, Instructor Name)

Instructor (Instructor ID, Instruct Name)

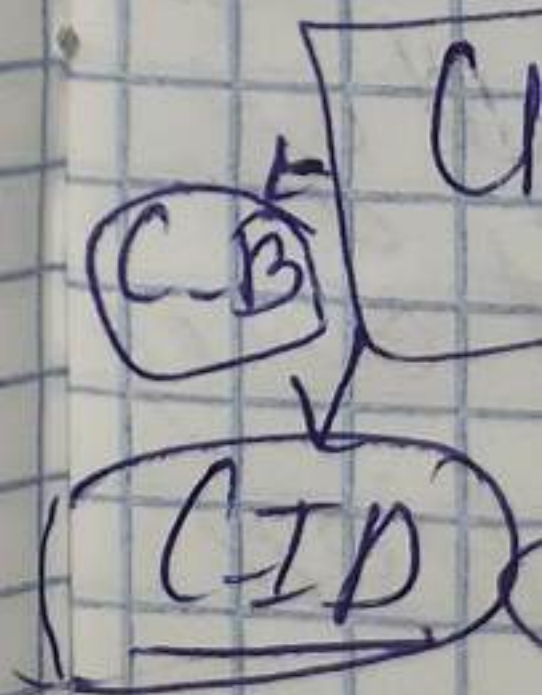
Room Building (Room, Building)

Class Schedule (Course ID, Timeslot, Instruct ID, Room)

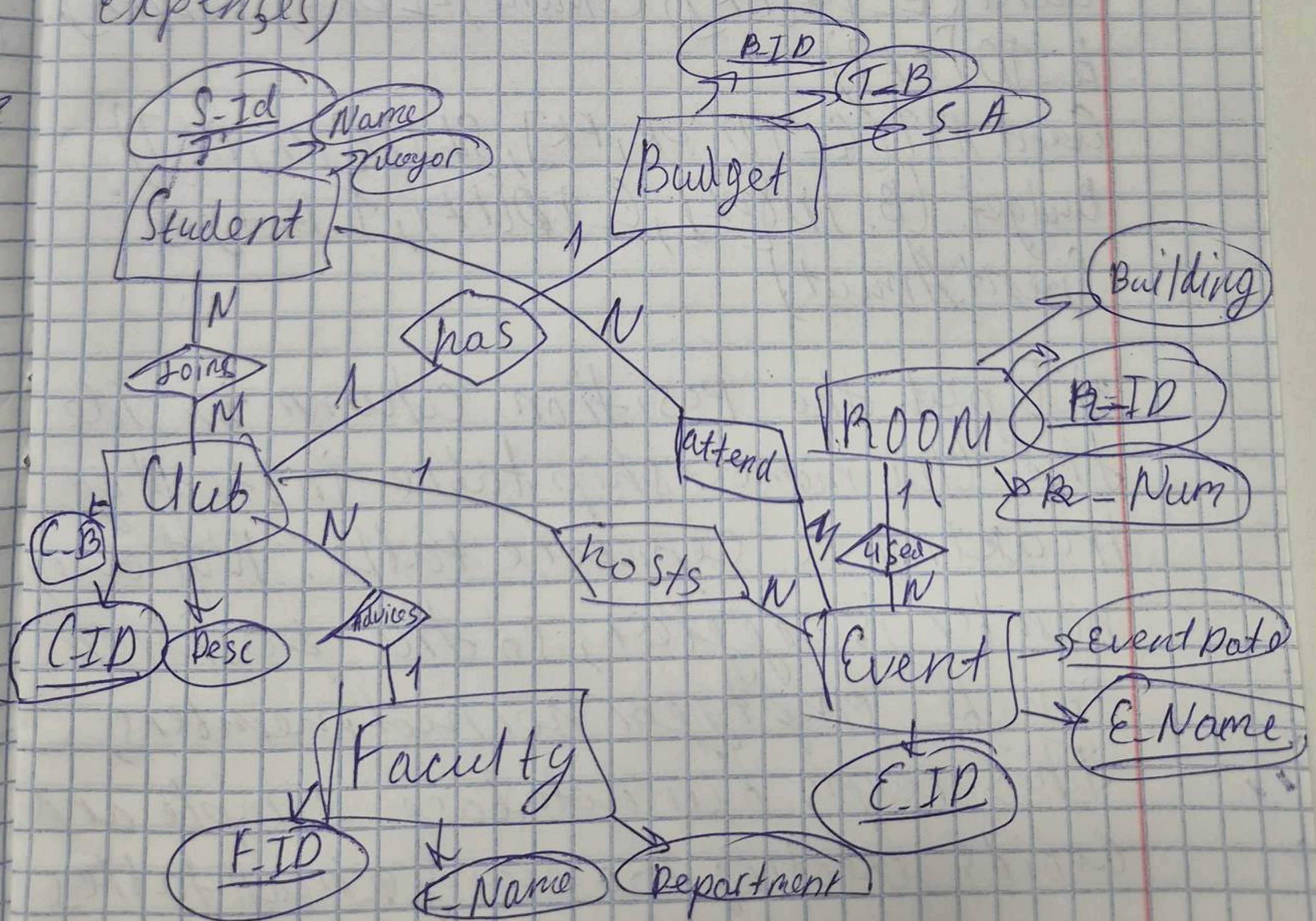
Student Enrollment (Student ID, Course ID, Timeslot)

5) If we split the tables correctly, we do not lose information. We can connect them back together using keys, and all rules are safely saved

5) St
Ch
Face
Even
B R O
B:00
Bup
Exp



5) Student (Student ID, Name, Major)
 Club (Club ID, Description, Faculty ID)
 Faculty (Faculty ID, Faculty Name, Department)
 Event (Event ID, Event name, Date, Club ID, ROOM ID)
 Room (ROOM ID, ROOM num, Building)
 Budget (Budget ID, Club ID, Total Amount, Expenses)



2) Normalized Relational Schema:

Student (S_ID [PK], S_Name, Major)
Faculty (F_ID [PK], F_Name, Department)
Club (C_ID [PK], C_Name, Desc, F_ID [FK])
ROOM (R_ID [PK], R_Num, Building)
Clubmembership (~~Student~~ S_ID [PK, FK], C_ID [PK, FK],
Position, Join Date)
Event (E_ID [PK], E_Name, E_Date, C_ID [FK],
R_ID [FK])
Event ATT (E_ID [PK, FK], S_ID [PK, FK])
Budget (B_ID [PK], C_ID [FK], T_Budget,
Spent Amount)

3) I put the position column inside the Club membership table instead of making a separate table. Not every member is officer, so the position can be empty for normal members. This keeps our database simple and avoids creating too many small tables.

4) 1) Officer
2) Li
week
3) Sh
for

- 4) 1) Find all students who are officers in the Computer Science Club.
- 2) List all events scheduled for the next week with their room reservations.
- 3) Show the total remaining budget for the Robotics Club.